Secure Login

Projektbeschreibung und Installationsanleitung

Sofia Wippel · 5BHWII · Fach: Informatik · Stand: 20. Oktober 2026

Teil 1 beschreibt das Projekt und seine Funktionsweise. **Teil 2** erklärt, wie es installiert und gestartet wird. Das Inhaltsverzeichnis ist anklickbar – ein Klick springt direkt zur gewünschten Stelle.

Diese Anleitung ist **nicht an einen bestimmten Rechner gebunden**. Sie funktioniert unter macOS, Linux und Windows und setzt nur die unten genannten Voraussetzungen voraus.

Inhaltsverzeichnis

	Teil 1 — Projektbeschreibung
1.1	Was ist das Projekt? →
1.2	Technische Eckdaten →
1.3	Funktionsumfang →
1.4	Projektstruktur →
1.5	Datenbank →
1.6	Die wichtigsten Abläufe →
	Teil 2 — Installationsanleitung
2.1	Voraussetzungen →
2.2	Installation in fünf Schritten →
2.3	Alternative: Betrieb mit Apache und XAMPP →
2.4	Die Seiten im Browser →
2.5	Wenn etwas nicht klappt →
2.6	Server beenden und wichtige Hinweise →

Teil 1 — Projektbeschreibung

1.1 Was ist das Projekt?

Das Projekt ist eine **vollständige Benutzeranmeldung mit Passwort-Zurücksetzung**, geschrieben in reinem PHP. Nutzer können sich registrieren, anmelden, ihr Konto mit einer zweiten Stufe absichern und ihr Passwort per E-Mail zurücksetzen.

Der Fokus liegt nicht auf der Optik, sondern auf **Sicherheit**: Passwörter werden nie im Klartext gespeichert, jedes Formular ist gegen Cross-Site-Request-Forgery geschützt, und wiederholte Anmeldeversuche werden begrenzt. Bewusst **ohne Framework** umgesetzt – rund **570 Zeilen** eigener Code, vollständig lesbar und ohne Abhängigkeiten.

Jede Seite in `public/` lädt dieselbe Basisdatei `src/app.php` und nutzt daraus Datenbankverbindung, Sitzungssteuerung, Formularschutz und die Anmeldeprüfung. Neue Seiten folgen demselben Muster: erst die Prüfung, dann die Ausgabe.

1.2 Technische Eckdaten

Eigenschaft	Details
Programmiersprache	PHP 8.2 oder neuer (Erweiterungen `pdo_mysql`, `sodium`, `curl`)
Datenbank	MariaDB 10.4 oder neuer (MySQL-kompatibel), Datenbank `auth_system`
Datenbankzugriff	PDO mit Prepared Statements (echtes Server-Seitiges Escaping)
Webserver	PHP-Entwicklungsserver (`php -S`) oder Apache/XAMPP
Sitzungen	Serverseitige PHP-Sessions, Cookie `login_project`
E-Mail-Versand	Resend-API (cURL) für die Reset-Links
Zwei-Faktor-Auth.	TOTP-Einmalcodes mit eigener Implementierung
Verschlüsselung	libsodium `secretbox` (AES-256) für 2FA-Geheimnisse
Frontend	Eigenes HTML/CSS, kein UI-Framework; QRCode.js
Codeumfang	21 Dateien, ca. 570 Zeilen eigener Code

1.3 Funktionsumfang

Funktion	Was passiert	Datei
Registrierung	E-Mail und Passwort (mind. 12 Zeichen), Dubletten abgefangen.	`register.php`
Login	Passwortprüfung, Versuchsbegrenzung, Umstellung alter Hash-Verfahren.	`login.php`
Konto-Seite	Angemeldete Nutzer sehen ihr Konto und richten 2FA ein.	`account.php`
Zwei-Faktor-Auth.	Einmalcode aus einer Authenticator-App.	`verify-2fa.php`
Passwort vergessen	Link per E-Mail, 30 Minuten gültig, einmalig benutzbar.	`forgot-password.php`

Passwort ändern	Neues Passwort setzen mit sicherer Token-Verwaltung.	`reset-password.php`
Abmelden	Sitzung wird zerstört, Sitzungs-Cookie gelöscht.	`logout.php`

1.4 Projektstruktur

```
login-project/
├── .env              Konfiguration (DB, E-Mail, APP_KEY) – nicht versioniert
├── .env.example      Vorlage aller Konfigurationswerte
├── .gitignore        schließt .env und .DS_Store aus
├── read_me.txt       Setup-Anleitung und Betriebshinweise
├── schema.sql        Datenbank-Schema (4 Tabellen)
├── db.php            veralteter Hilfscode, wird nicht verwendet
│
├── src/              Anwendungslogik – NICHT öffentlich
│   ├── app.php       Config, Datenbank, Session, CSRF, Auth, Rate-Limit
│   ├── totp.php      2FA: Base32, TOTP-Berechnung, Verschlüsselung
│   └── mail.php      E-Mail-Versand über die Resend-API
│
└── public/           Web-Root: der einzige öffentliche Bereich
    ├── index.php     Weiterleitung zu Login oder Konto
    ├── login.php     Anmeldung
    ├── register.php  Registrierung
    ├── account.php   Kontoverwaltung und 2FA-Einrichtung
    ├── verify-2fa.php, forgot-password.php, reset-password.php, logout.php
    └── assets/       style.css, account.js, qrcode.min.js
```

Das ist der wichtigste Punkt: Nur `public/` wird als Web-Root ausgegeben. Konfiguration und Anwendungslogik liegen bewusst *außerhalb* und können über die Browser-Adresse nicht abgerufen werden.

1.5 Datenbank

Tabelle	Inhalt
`users`	ID, E-Mail (eindeutig), Passwort-Hash, letzte Anmeldung, Erstellungsdatum
`password_resets`	Benutzer-ID, Hash des Reset-Tokens, Ablaufzeit (30 Minuten)
`user_totp`	Benutzer-ID, Nonce und Chiffretext des 2FA-Geheimnisses, letzter Zeitschritt
`auth_rate_limits`	Kennung, Anzahl Versuche, Ablaufzeitpunkt

Alle Zeitstempel liegen in UTC – so funktionieren Ablaufzeiten unabhängig von der Systemzeitzone.

1.6 Die wichtigsten Abläufe

Anmeldung

1. CSRF-Token und Versuchsbegrenzung prüfen, Benutzer per Prepared Statement laden.
2. Passwort mit `password_verify` prüfen, altes Hash-Format automatisch umstellen.
3. Ist 2FA aktiv → Code-Seite, sonst Sitzung eröffnen.

Zwei-Faktor-Authentifizierung

1. Zufälliges Geheimnis erzeugen, QR-Code im Browser daraus erstellen.
2. Nach Bestätigung wird es verschlüsselt gespeichert.
3. Bei jeder Anmeldung wird zusätzlich ein zeitbasierter Einmalcode verlangt; bereits genutzte Zeitschritte werden gesperrt.

Passwort zurücksetzen

1. Token erzeugen, hashen und mit 30 Minuten Ablaufzeit speichern.
2. Link per E-Mail verschicken – bei Fehler wird der Token wieder entfernt.
3. Beim Setzen wird der Token in einer Transaktion gesperrt geprüft, danach werden alle offenen Reset-Tokens gelöscht.

Teil 2 — Installationsanleitung

2.1 Voraussetzungen

Für das Projekt wird Folgendes benötigt:

Voraussetzung	Anforderung
PHP	Version 8.2 oder neuer, mit den Erweiterungen `pdo_mysql`, `sodium` und `curl`
Datenbank	MariaDB 10.4 oder neuer, alternativ MySQL 5.7 oder neuer
Webserver	Entweder der mitgelieferte PHP-Entwicklungsserver oder Apache (z. B. XAMPP)
Browser	Jeder aktuelle Browser
E-Mail-Dienst	Für den Passwort-Reset wird ein Resend-Konto mit verifizierter Absenderadresse benötigt

Wer XAMPP nutzt, bringt PHP, Apache und MariaDB bereits mit. Wer mit der Kommandozeile arbeitet, installiert die Bestandteile einzeln, zum Beispiel mit `brew install php mariadb` (macOS) oder `apt install php php-mysql mariadb-server` (Debian/Ubuntu).

2.2 Installation in fünf Schritten

Im Folgenden steht `/pfad/zum/login-project` für den Ordner, in dem das Projekt liegt. Alle Befehle werden in diesem Ordner ausgeführt.

Schritt 1 — Datenbank starten

Je nach Installation:

```
# XAMPP: Dienst im Steuerungsfeld starten
# macOS mit Homebrew:
brew services start mariadb
# Debian / Ubuntu:
sudo systemctl start mariadb
```

Schritt 2 — Datenbank und Tabellen anlegen

Der Befehl liest `schema.sql` ein und legt die Datenbank samt vier Tabellen an. Vorhandene Konten bleiben dabei erhalten, weil die Anweisungen „erst anlegen, falls nicht vorhanden" verwenden. Der Datenbank-Benutzer und das Passwort müssen zu `.env` passen (Schritt 3).

```
cd /pfad/zum/login-project
mariadb -u root < schema.sql
# oder: mysql -u root -p < schema.sql
```

Schritt 3 — Konfiguration anlegen

Die Datei `.env` enthält alle Zugangsdaten. Sie wird aus der Vorlage erzeugt und anschließend befüllt:

```
cp .env.example .env
```

Variable	Bedeutung
`DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	Zugangsdaten zur Datenbank. `DB_NAME` muss `auth_system` entsprechen.
`APP_URL`	Adresse, unter der das Projekt erreichbar ist, z. B. `http://127.0.0.1:8081`. Wird für die Reset-Links verwendet.
`RESEND_API_KEY`	API-Schlüssel des E-Mail-Dienstes.
`RESEND_FROM`	Absenderadresse einer bei Resend verifizierten Domain, z. B. `Login <login@example.com>`.
`APP_KEY`	32 zufällige Bytes, Base64-kodiert. Wird zum Verschlüsseln der 2FA-Geheimnisse gebraucht.

Ein passender `APP_KEY` wird so erzeugt:

```
php -r 'echo base64_encode(random_bytes(32)), PHP_EOL;'
```

> **Achtung:** Der `APP_KEY` darf nach dem ersten Start nicht mehr geändert werden – sonst lassen sich die gespeicherten 2FA-Geheimnisse nicht mehr entschlüsseln. Die Datei `.env` gehört nicht in die Versionsverwaltung; sie ist bereits in der `.gitignore` ausgenommen.

Schritt 4 — Server starten

Der eingebaute PHP-Server wird im Projektordner gestartet. Entscheidend ist `-t public`: damit wird nur der Ordner `public/` als Web-Root ausgegeben.

```
cd /pfad/zum/login-project
php -S 127.0.0.1:8081 -t public
```

Die Adresse lässt sich frei wählen. Auf einem belegten Port einfach eine andere Nummer verwenden.

Schritt 5 — Browser öffnen

```
http://127.0.0.1:8081
```

Die Startseite leitet automatisch zur Anmeldung weiter. Ein Konto lässt sich über **„Konto erstellen"** anlegen.

2.3 Alternative: Betrieb mit Apache und XAMPP

Wer statt des Entwicklungs servers einen echten Webserver einsetzen möchte, trägt das Projekt als **Virtual Host** ein. Grundsatz bleibt: Das Document-Root muss auf den Ordner `public/` zeigen, damit `.env` und `src/` nicht

öffentlich sind.

```
<VirtualHost *:80>
    DocumentRoot "/pfad/zum/login-project/public"
    <Directory "/pfad/zum/login-project/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

Danach Apache neu laden und das Projekt über die konfigurierte Adresse aufrufen.

> **Wichtig:** Wird der gesamte Projektordner als Document-Root verwendet, sind Konfiguration und Anwendungscode unter `/.env` und `/src/` im Browser abrufbar. Der Zugriff auf die Datei `.env` liefert dann den E-Mail-Schlüssel und den Verschlüsselungsschlüssel aus.

2.4 Die Seiten im Browser

Adresse	Seite
`/`	Weiterleitung zur Anmeldung bzw. zum Konto
`/login.php`	Anmeldung
`/register.php`	Registrierung
`/account.php`	Kontoverwaltung (nur angemeldet)
`/verify-2fa.php`	Prüfung des Einmalcodes
`/forgot-password.php`	Reset-Link anfordern
`/reset-password.php?token=…`	Neues Passwort setzen

2.5 Wenn etwas nicht klappt

Meldung / Symptom	Ursache und Lösung
`Datenbankverbindung fehlgeschlagen`	Der Datenbankdienst läuft nicht, oder die Zugangsdaten in `.env` stimmen nicht.
Seite bleibt weiß / 404	Der Server läuft nicht oder zeigt auf den falschen Ordner → `php -S 127.0.0.1:8081 -t public`
Tabelle „users" fehlt	Schema nicht eingelesen → `mariadb -u root < schema.sql`
„E-Mail-Versand ist nicht eingerichtet"	In der `.env` fehlen `RESEND_API_KEY` oder `RESEND_FROM`
Reset-Link kommt nicht an	Die Absenderadresse ist bei Resend nicht verifiziert, oder `APP_URL` stimmt nicht.
„Zu viele Versuche"	Die Versuchsbegrenzung greift → warten oder den Eintrag in `auth_rate_limits` löschen
„2FA-Schlüssel nicht entschlüsselbar"	Der `APP_KEY` wurde geändert → Originalwert wiederherstellen
Port bereits belegt	Anderen Port verwenden, z. B. `php -S 127.0.0.1:8082 -t public`

2.6 Server beenden und wichtige Hinweise

Der Entwicklungsserver läuft im Vordergrund und wird im Terminal mit `Strg` + `C` beendet. Ein per `brew` oder `systemctl` gestarteter Datenbankdienst wird entsprechend mit `brew services stop mariadb` beziehungsweise `sudo systemctl stop mariadb` gestoppt.

- `.env` **nie weitergeben.** Die Datei enthält den API-Schlüssel des E-Mail-Versands und den Verschlüsselungsschlüssel für die 2FA-Geheimnisse. Genau deshalb steht sie in der `.gitignore`.
- **Das Document-Root muss auf** `public/` **zeigen.** Nur so bleiben Konfiguration und Anwendungscode vor dem Browser verborgen.
- **Datenbank zuerst, dann Server.** Ohne laufende Datenbank schlägt jeder Seitenaufruf fehl.
- **Ablaufzeiten gelten ab dem Zeitpunkt der Anfrage.** Der Token eines Passwort-Resets ist 30 Minuten lang gültig und danach ungültig – auch wenn der Link noch im Postfach liegt.